

CUSTOMER FEATURE GUIDE

Hermes

The application operating layer for Kubernetes —
Every Application. One Door.

Discover, organize, secure, and launch every application running across your clusters
through a single intelligent gateway.

by ZyvorAI Labs

What is **Hermes**?

Hermes turns Kubernetes from an infrastructure platform into an application platform. It continuously discovers every dashboard, API, and internal tool running in your cluster, recognizes what each one is, monitors its health, and gives users a permanent front door to launch software — no more kubectl port-forward, bookmark sprawl, or namespace archaeology. Think macOS Launchpad and Spotlight, but for your clusters.

40+

REST & gateway endpoints

11

Nebula UI pages

8

Zeus AI insight APIs

15+

recognized platform tools

What's inside this guide

Getting started — access & first workflows	start here
1 · Continuous Discovery	5 features
2 · Recognition & Catalog	5 features
3 · Universal Gateway	5 features
4 · Nebula Launchpad UI	6 features
5 · Spotlight Search	4 features
6 · Zeus AI Intelligence	5 features
7 · Health & Diagnostics	5 features
8 · Application Graph	4 features
9 · Multi-Cluster & Teams	5 features
10 · Security & Access	6 features
11 · Sharing, Audit & Memory	5 features
12 · Deployment & Operations	5 features
Support & contact	→

Access & first workflows

How to reach Hermes and get your first result. Assumes it has been deployed for you.

How to access it

WEB

Nebula launchpad UI on NodePort ****31847**** — open `http://localhost:31847` (or `http://<node-ip>:31847`). Press `⌘K` / `Ctrl+K` for Spotlight to search and launch any app through the gateway. Key pages: `/` (Command Deck), `/apps`, `/cluster`, `/discovery`, `/health`, `/graph`, `/teams`, `/federated`, `/activity`.

CLI

There is no dedicated end-user CLI — Hermes is operated with `kubectl` / `helm` plus repo scripts (`./scripts/deploy-remote.sh <host> <user>`, `./scripts/configure-llm-remote.sh`, `./scripts/setup-ollama-remote.sh`, `./scripts/e2e-deploy-verify.sh http://host:31847`). Every action is also scriptable over REST with `curl`.

API

REST + WebSocket surface under `http://localhost:31847/api/v1`. Public (no auth): `GET /healthz`, `GET /metrics`, `/auth/*`, `/api/v1/ws-echo`. Example: `curl -s http://localhost:31847/api/v1/apps | jq '.[0]'`. Embedded Zeus AI insights live under `/api/v1/insights/*` and `/api/v1/search/llm`.

LOGIN

Auth is disabled unless `HERMES_AUTH_MODE` is set. With `api_key`, send `Authorization: Bearer <key>`; with `oidc`, sign in at `/auth/login` (redirects to your IdP, sets a session cookie). `GET /auth/me` returns the current user, groups, workspaces, and allowed actions.

NEEDS

A running Kubernetes cluster (Hermes installs via Helm); optionally set an OpenAI-compatible LLM (`HERMES_LLM_API_URL` + `HERMES_LLM_API_KEY`) to unlock Zeus AI natural-language answers — rule-based mode works with no key.

Your first workflows



Install and open Hermes

1. Install the chart: `helm install hermes oci://ghcr.io/hypersdk/charts/hermes --version 0.2.0 --namespace hermes-system --create-namespace`.
2. Wait for rollout: `kubectl -n hermes-system rollout status deployment/hermes`.
3. Open the Nebula UI at `http://<node-ip>:31847` — discovery starts automatically across every namespace.



Review the discovery queue and publish an app

1. Let discovery run (`discoverAll: true` by default) so newly found Services land in the queue.
2. Open `/discovery` in the UI and review candidates ranked by Zeus AI publish suggestions.
3. Click Publish on the right services (or `POST /api/v1/discovery/publish/{namespace}/{slug}`; bulk with `/api/v1/discovery/publish-namespace/{namespace}`).
4. The app now appears on Home Quick Launch and the `/apps` catalog.



Launch an app with zero port-forward

1. Open the UI and press `⌘K` / `Ctrl+K` to open Spotlight.
2. Type a few letters of the app name, namespace, or category (e.g. `gra` for Grafana).
3. Press Enter to launch it through the gateway at `/launchpad/apps/{slug}` — WebSockets, SSE, and gRPC proxy transparently, no `kubectl port-forward`.



Enrich a custom in-house app

1. Add `hermes.zyvor.dev/*` annotations to the Kubernetes Service — e.g. `hermes.zyvor.dev/name`, `/icon`, `/owner`, `/category`, `/depends-on`.
2. Set `hermes.zyvor.dev/published: "true"` to publish it immediately, or leave it for the discovery queue.
3. The app renders with a polished name, icon, owner, and dependency edges on `/apps` and `/graph`.



Diagnose an unhealthy app

1. Open `/health` and read the attention queue with its Zeus AI summary.
2. Click Diagnose on a failing service to open the global Diagnosis drawer.
3. Read the route lens, friendly probe summary (expand for raw detail), suggested `kubectl` commands, and Zeus AI remediation narrative (`GET /api/v1/apps/{id}/diagnosis` + `/insight`).



Enable Zeus AI natural-language mode

1. Create the LLM secret: `kubectl create secret generic hermes-llm --from-literal=apiKey="sk-..." -n hermes-system`.
2. Upgrade with LLM settings: `helm upgrade hermes oci://ghcr.io/hypersdk/charts/hermes --reuse-values --set server.llm.apiUrl="https://api.openai.com/v1" --set server.llm.existingSecret="hermes-llm" --set server.llm.model="gpt-4o-mini" -n hermes-system`.
3. Confirm the active mode: `GET /api/v1/insights/status` (or Spotlight `ai status`) should report `defaultSource: llm`.

Continuous Discovery

Hermes watches the cluster and builds a living application catalog for you — no manual registration.



Cluster-Wide Service Discovery

Continuously watches Kubernetes Services across every namespace and adds them to the catalog automatically.

Stop maintaining spreadsheets of service URLs — the cluster documents itself.

How · Automatic — the Go controller watches Services cluster-wide (`discoverAll: true`). Browse results at `/cluster` in the UI or `GET /api/v1/apps` .



Ingress & Gateway API Routes

Discovers routing from Ingress objects and Gateway API HTTPRoutes, including ingress host metadata on each app.

See the real public entry point for an app, not just its internal service name.

How · Automatic during discovery; the ingress host shows on the app card at `/apps/:id` and in `GET /api/v1/apps/{id}` .



Service Mesh Route Discovery

Reads Istio VirtualServices and Linkerd annotations to map mesh-based routing into the catalog.

Mesh-routed apps become as discoverable as everything else.

How · Automatic for Istio VirtualServices and Linkerd annotations; filter mesh-routed apps on `/graph` or via `GET /api/v1/graph` .



Automatic Catalog Refresh

Correlates EndpointSlice readiness and updates the catalog live as the cluster changes.

The catalog is always current without any human upkeep.

How · Automatic — the controller correlates EndpointSlice readiness; no action needed. Live counts at `GET /api/v1/cluster/summary` .



Discovery Queue

Surfaces newly found but unpublished services in a dedicated review queue separate from the live launchpad.

Nothing gets a front door by accident — you decide what goes live.

How · Web UI `/discovery` ; REST `GET /api/v1/discovery` .

Recognition & Catalog

Hermes identifies popular software automatically and enriches every app with human-friendly metadata.

Recognized tool	Example signals
Grafana	service name, labels, port 3000
Prometheus	prom/prometheus image patterns
Argo CD	argocd namespace + service names
Harbor	Harbor service patterns
Zeus OS / v9s	Zeus, ConsoleHub, platform labels
Longhorn, Guacamole, Hubble, Traefik	known platform signatures



Signature Recognition Engine

Matches service names, labels, images, and ports against known signatures to identify popular tools automatically.

Apps show up with the right name, category, and icon — usually with zero configuration.

How · Automatic during discovery; the recognized name, category, and icon appear on `/apps` cards and in `GET /api/v1/apps`.



Annotation Enrichment

Optional `hermes.zyvor.dev/*` annotations set display name, slug, category, icon, owner, dependencies, port, scheme, and path.

Custom in-house apps get the same polished catalog treatment as off-the-shelf tools.

How · Add `hermes.zyvor.dev/*` annotations to the Service (e.g. `hermes.zyvor.dev/name`, `/icon`, `/owner`, `/category`) — see `annotations.md`.



Environment Inference

Labels each app production, staging, or development from annotations, or infers it from the namespace name.

Instantly tell prod from dev without asking anyone.

How · Set `hermes.zyvor.dev/environment`, or let Hermes infer it from the namespace name (`*-prod`, `staging`, `dev`); filter by workspace in the navbar.



Published App Catalog

A unified `/apps` catalog of published applications with categories, search, sort, filter, and view toggles.

One browsable home for every tool your team actually uses.

How · Web UI `/apps`; REST `GET /api/v1/catalog`.



Catalog Export

Download the full catalog as JSON via the API or the Cluster page action menu.

Feed your app inventory into other systems or keep an auditable snapshot.

How · Cluster page ActionMenu → export, or `GET /api/v1/catalog/export` .

Universal Gateway

Every published workload gets a permanent, predictable front door through one reverse proxy.



Stable Launchpad URLs

Proxies apps at predictable paths like `/launchpad/apps/{slug}` and `/a/{namespace}/{slug}`, with legacy aliases.

Links survive pod reschedules and namespace moves — no bookmark ever breaks.

How · Click an app in the launchpad, or hit `/launchpad/apps/{slug}` or `/a/{namespace}/{slug}` directly (legacy `/apps/{slug}` alias supported).



WebSocket Tunneling

Proxies WebSocket upgrades for live dashboards like Grafana, Argo CD, VS Code Server, Open WebUI, and Jupyter.

Interactive tools work fully through the gateway, not just static pages.

How · Automatic — WebSocket upgrades proxy over the same launchpad paths; no configuration.



SSE, HTTP/2 & gRPC Pass-Through

Supports Server-Sent Events streaming, HTTP/2, and gRPC content types with native backend connection pools.

Modern streaming and RPC apps proxy correctly and efficiently.

How · Automatic — the gateway proxies any HTTP method and content type on `/launchpad/apps/{slug}`.



Readiness-Gated Proxying

Blocks the proxy with a clear error when a backend has no ready endpoints.

Users never get silently routed to a dead workload.

How · Automatic — the proxy returns 403/502 when no ready endpoints exist; verify with `curl -o /dev/null -w '%{http_code}\n' $BASE/a/{namespace}/{slug}`.



Zero Port-Forwarding

Replaces the kubectl port-forward workflow with search, click, launch, work.

Developers reach any tool in seconds without touching kubectl.

How · Open the app from Spotlight (`⌘K`) or the `/apps` catalog — no `kubectl port-forward` required.

SECTION 4

Nebula Launchpad UI

A macOS-inspired glass launchpad that looks like applications, not namespaces and pods.



Command Deck Home

A hero with fleet snapshot, a 3-metric summary, Quick Launch tiles, and Mission Control accordions.

The one screen that answers what's running, what's healthy, and what to open.

How · Web UI / (the Command Deck home page).



Cluster Catalog View

Every service grouped by namespace in collapsible sections with human-readable health hints.

Explore the whole cluster's apps without a single `kubectl` get.

How · Web UI /cluster ; backing data from `GET /api/v1/apps` and `GET /api/v1/cluster/summary`.



Spaces Organization

Organizes published apps into category-based spaces for browsing.

Find the monitoring stack or CI tools as a group, not a flat list.

How · Web UI Spaces view (category-grouped catalog); categories come from signature recognition or `hermes.zyvor.dev/category`.



Liquid Glass Design System

A tiered translucent glass interface with shared page frames, skeleton loading, and empty states.

A fast, polished, consistent experience across every page.

How · Applies automatically across all Nebula pages — no user action needed.



Keyboard Navigation & Shortcuts

Arrow-key and Enter navigation in Spotlight plus a global keyboard shortcuts help panel.

Power users move through the whole app without a mouse.

How · Press `⌘K` / `Ctrl+K` to open Spotlight, arrow keys + Enter to navigate; open the global shortcuts help panel from the UI.



Responsive Mobile Layouts

Compact workspace switcher, stacked toolbars, and scrollable category chips below 768px.

Launch and check apps from a phone, not just a desktop.

How · Automatic below 768px — open any Nebula page on a phone; toolbars stack and category chips scroll.

Spotlight Search

A Command-K palette that finds apps, pages, and answers the way Spotlight finds anything.



Full-Text Catalog Search

Type a few letters to match apps by name, namespace, or category across the whole cluster.

Type `gra`, get `Grafana` — no memorizing service DNS names.

How · Press `⌘K` and type, or call `GET /api/v1/search?q=grafana`.



Rule-Based Intent Search

Parses queries into intent so a namespace or category returns exactly the right apps.

Ask for what you mean and get relevant results, not keyword noise.

How · Spotlight parses intent automatically; direct API `GET /api/v1/search/intent?q=`.



Natural-Language Search

An LLM-powered search endpoint answers plain-language queries, with rule-based fallback when no LLM is set.

Ask `unhealthy apps in monitoring in English` and get a real answer.

How · `GET /api/v1/search/llm?q=unhealthy+apps` — falls back to rules when no LLM is configured.



Spotlight AI Commands

Built-in commands like `explain`, `why`, `diagnose <app>`, `suggest publish`, `ns insight`, and `ai status`.

Run diagnostics and get fleet answers straight from the search bar.

How · In Spotlight (`⌘K`) type `explain`, `why`, `diagnose <app>`, `suggest publish`, `ns insight`, `graph insight`, `owner insight`, or `ai status`.

Zeus AI Intelligence

Natural-language insight over your application catalog — with or without an external LLM.



Fleet Insight

Summarizes overall cluster application health and highlights the services that need attention.

Understand the state of everything at a glance, in plain language.

How · Home / Health fleet panel with focus chips; REST GET `/api/v1/insights/fleet`.



Per-App Insight

Generates a diagnosis and remediation narrative for a single application.

Know why an app is unhealthy and what to do about it.

How · App detail `/apps/:id`, the Diagnose drawer, or Inspector AI tab; REST GET `/api/v1/apps/{id}/insight`.



Publish Suggestions

Ranks discovery-queue candidates so you know which services are worth publishing.

Curate the launchpad quickly instead of reviewing everything by hand.

How · Suggestions on the `/discovery` page; REST GET `/api/v1/insights/discovery`; Spotlight suggest publish.



Namespace, Owner & Graph Insight

Health rollups per namespace and per team, plus topology insight for unresolved dependency links.

Slice fleet intelligence by the dimension you care about.

How · REST GET `/api/v1/insights/namespace/{ns}`, `/insights/owner/{owner}`, `/insights/graph`; Spotlight ns insight, owner insight, graph insight.



Dual-Mode LLM + Rules

Uses an OpenAI-compatible LLM when configured and falls back to deterministic rules otherwise; a status endpoint reports which is active.

Full intelligence with no external API required — connect an LLM only if you want richer answers.

How · Set `HERMES_LLM_API_URL`, `HERMES_LLM_API_KEY`, `HERMES_LLM_MODEL`; check the active mode via GET `/api/v1/insights/status` or Spotlight ai status.

Zeus AI works out of the box in **rule-based mode** with no API key. Point it at OpenAI, an OpenAI-compatible endpoint, or a local Ollama install (helper script included) to unlock richer natural-language answers.

Health & Diagnostics

Real-time health for every app, translated from raw probe errors into answers humans can act on.



Real-Time Health Status

Classifies every app as Healthy, Degraded, or Offline from endpoint readiness and backend probes.

Know an app works before you send a teammate to it.

How · Status shows on `/apps` and `/health` cards; REST `GET /api/v1/health/apps`.



Attention Queue

The Health page lists exactly the services that need action, with an AI summary.

Focus on what's broken instead of scanning a wall of green.

How · Web UI `/health` (attention queue with Zeus AI summary).



Diagnose Drawer

A global drawer showing route lens, suggested kubectl commands, probe errors, and Zeus AI insight for any app.

Go from something's wrong to a fix without leaving the launchpad.

How · Click Diagnose or Inspect route on any app to open the global drawer; REST `GET /api/v1/apps/{id}/diagnosis`.



Friendly Probe Summaries

Maps raw errors like context deadline exceeded into short summaries with an expandable technical view.

Everyone understands the status; operators can still see the raw detail.

How · Automatic on cards — a short summary plus endpoint chip; expand technical details on the card or Diagnose drawer for the raw message.



Fleet Health Indicators

Home hero and navbar health chip reflect the live fleet health percentage.

A single glance tells you how the whole cluster is doing.

How · Home hero + navbar health chip; underlying data from `GET /api/v1/health/apps`.

Application Graph

See how your applications actually connect, and where a dependency is failing.



Dependency Topology

A `/graph` page rendering nodes and edges of app-to-app dependencies with interactive filters.

Understand what talks to what before you change or debug something.

How · Web UI `/graph` ; REST `GET /api/v1/graph` (returns `nodes` and `edges`).



Mesh Route Filters

Filter the graph by service-mesh routing to focus on mesh-connected apps.

Isolate the mesh view when tracing east-west traffic.

How · Use the filter controls on the `/graph` page to scope to mesh-routed apps.



AI Focus Chips

Zeus AI highlights unresolved or broken dependency links directly on the graph.

The graph points you at the weak link instead of leaving you to hunt.

How · Shown on `/graph` ; REST `GET /api/v1/insights/graph` or Spotlight `graph insight` .



Declared Dependencies

Reads depends-on annotations to draw explicit relationships like Grafana to Prometheus to Loki.

Your intended architecture shows up as a real, navigable map.

How · Set `hermes.zyvor.dev/depends-on: "prometheus, loki"` on the Service; edges render on `/graph` .

Multi-Cluster & Teams

Merge catalogs across clusters and organize applications by the teams that own them.



Federated Catalog

Merges catalogs from remote Hermes clusters into one view with remote health.

One launchpad spanning dev, prod, edge, and cloud clusters.

How · Web UI `/federated` ; REST `GET /api/v1/catalog/federated` . Configure peers via `HERMES_FEDERATED_CLUSTERS` (JSON) / `cluster.federated` in values.yaml.



Cross-Cluster Publishing

Publish apps, bulk-publish namespaces, and set recommendations on federated clusters via the API.

Curate remote clusters without leaving your home Hermes.

How · REST `POST /api/v1/federation/publish/{cluster_id}/{id}` , `/federation/publish-namespace/{cluster_id}/{namespace}` , `PUT /federation/recommended/{cluster_id}/{id}` .



Team Ownership Views

The Teams page groups applications by owner metadata with per-team health rollups.

Instantly see which team owns what — and how their apps are doing.

How · Web UI `/teams` ; REST `GET /api/v1/owners` ; set the owner with `hermes.zyvor.dev/owner` .



Workspace Filtering

Filter the catalog by environment workspace (dev / staging / production), with permissions via OIDC groups.

Each person sees the environment slice that's relevant to them.

How · Use the navbar workspace chips / compact switcher; REST `GET /api/v1/workspaces` . OIDC groups gate which workspaces a user sees.



Cluster Registry

Lists local and federated clusters and reports remote health and RBAC.

A single inventory of every cluster Hermes knows about.

How · REST `GET /api/v1/clusters` ; remote RBAC checked via `GET /api/v1/federation/rbac/{cluster_id}` .

Security & Access

Internal stays internal until you say otherwise — with auth, RBAC, and network controls.



Explicit Publish Control

autoPublish defaults to off, so discovered apps require an explicit publish action or published annotation.

No workload is ever exposed by accident.

How · `autoPublish: false` by default — publish via `/discovery` (or `POST /api/v1/discovery/publish/{id}`) or set `hermes.zyvor.dev/published: "true"`.



API Key & OIDC Auth

Optional authentication via static API key or OIDC single sign-on with session cookies.

Gate the whole surface behind your existing identity provider.

How · Set `HERMES_AUTH_MODE=api_key` (send `Authorization: Bearer <key>`) or `oidc` (sign in at `/auth/login`, session cookie).



Role Rules & Admin Groups

Configurable role rules plus OIDC admin groups and admin users for elevated actions.

Give the right people admin power without sharing a superuser.

How · Configure `HERMES_ROLE_RULES`, `HERMES_ADMIN_GROUPS`, and admin users; `GET /auth/me` reports the caller's allowed actions.



Kubernetes SAR RBAC

Optional SubjectAccessReview enforcement so users only see and reach apps they're allowed to.

Reuse your cluster's own RBAC as the source of truth for access.

How · Enable SAR enforcement via Helm values; API list and gateway routes then filter apps by SubjectAccessReview.



Namespace Allow-List & NetworkPolicy

Scope discovery to specific namespaces and restrict pod-to-pod traffic with a bundled NetworkPolicy template.

Contain Hermes to exactly the blast radius you intend.

How · Set `HERMES_ALLOWED_NAMESPACES` to scope discovery; enable the bundled NetworkPolicy Helm template to restrict pod-to-pod traffic.



Federation Trust & RBAC Sync

Forwards identity groups across clusters with trust headers so RBAC stays consistent in federation.

Access rules follow the user across every federated cluster.

How · Automatic trust-header group forwarding to federated peers; verify remote access with `GET /api/v1/federation/rbac/{cluster_id}`.

Sharing, Audit & Memory

Time-limited share links, a full audit trail, and personal launch memory.



Time-Limited Share Links

Create read-only share tokens with a TTL that proxy directly to an app; admins can list and revoke them.

Hand someone temporary access to a dashboard without granting an account.

How · Create from `/apps/:id`; REST `POST /api/v1/shares` (body `appId`, `ttlHours`); links proxy at `/s/{token}`; revoke with `DELETE /api/v1/shares/{token}` (admins list all via `/api/v1/shares/all`).



Activity Audit Log

Records launches, share access, and admin actions, viewable on the Activity page with CSV export.

Always know who opened what, and when.

How · Web UI `/activity` (with CSV export); REST `GET /api/v1/audit?limit=`.



Federated Audit Aggregation

Aggregates audit events across federated clusters into a single feed.

One compliance view spanning every cluster.

How · REST `GET /api/v1/audit/federated?limit=`; Spotlight `activity insight` and `GET /api/v1/insights/activity` summarize patterns.



Favorites

Pin the apps you use most for one-click access.

Your daily tools are always a single click away.

How · Pin from the app card; REST `PUT /api/v1/favorites/{id}` / `DELETE /api/v1/favorites/{id}` (`GET /api/v1/favorites` lists them).



Recents & Recommendations

Tracks recently opened apps and surfaces team-recommended picks on Home.

The launchpad learns your workflow and highlights what matters.

How · Recents auto-tracked on launch (`GET /api/v1/recents`); team picks via `hermes.zyvor.dev/recommended: "true"` or `PUT /api/v1/recommended/{id}` (`GET /api/v1/recommended`).

Deployment & Operations

Ship Hermes with a Helm chart, monitor it with Prometheus, and run it on a lean stack.



Helm Chart Install

One helm install deploys the Go controller and Rust server, published as an OCI Helm chart.

Stand up the whole application layer in a single command.

How · `helm install hermes oci://ghcr.io/hypersdk/charts/hermes --version 0.2.0 --namespace hermes-system --create-namespace .`



Prometheus Metrics

Exposes a `/metrics` endpoint with app totals, audit counters, and uptime, plus a `/api/v1/stats` summary.

Wire Hermes straight into your existing monitoring.

How · Scrape `GET /metrics` (`hermes_apps_total`, audit counters, uptime); catalog summary at `GET /api/v1/stats`.



Lightweight SQLite Catalog

Controller and server share a single SQLite catalog on a PVC — no external database to run.

Minimal moving parts and dependencies to operate.

How · Automatic — the controller and server share a SQLite catalog on a PVC; no external database to configure.



Remote Deploy Tooling

Scripts for remote k3s deploy, LLM configuration, Ollama setup, and end-to-end verification.

Get from git clone to a verified running cluster fast.

How · Run `./scripts/deploy-remote.sh <host> <user>`, `./scripts/configure-llm-remote.sh <host> <user>`, and `./scripts/setup-ollama-remote.sh <host> <user>`.



Demo Apps & Smoke Tests

Ships demo Grafana and Prometheus plus smoke and E2E verification scripts.

Prove the install works end to end in minutes.

How · Run `./scripts/smoke-test.sh` locally and `./scripts/e2e-deploy-verify.sh http://host:31847` against a deploy; UI Playwright via `HERMES_E2E_BASE=http://host:31847 npm run test:e2e`.

Support & next steps

Support & contact

Hermes is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

Good to know: Feature availability reflects Hermes v0.2. Some capabilities are roadmap-only and not yet shipped, including in-cluster mesh policy editing and federated activity export webhooks. Discovery covers Kubernetes Services, Ingress/Gateway API routes, and Istio/Linkerd mesh routes today; discovery of arbitrary custom resources is on the roadmap. The multi-cluster registry is an early single-cluster-first MVP. Hermes ships with a 30-day trial license (HMAC-SHA256 enforced); continued use requires a commercial license from sales@zyvor.dev.

Hermes

Discover, organize, secure, and launch every application running
across your clusters through a single intelligent gateway.

info@zyvor.dev · zyvor.dev